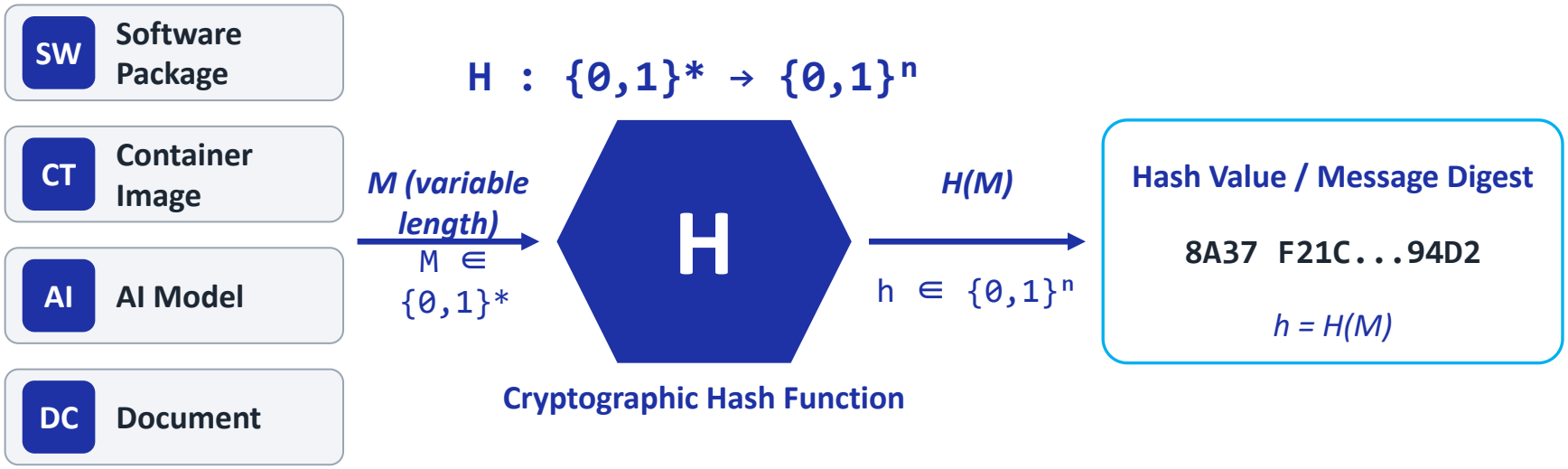
MODULE 2 · LECTURE 8

CS 6530 Applied Cryptography

Cryptographic Hash Functions

Module 2 – Symmetric Cryptography, Hashes and Authentication

What is a hash? intuition + formal model



Variable-length data → Fixed-length digest; Deterministic . Efficient . Unkeyed

ENGINEERING INSIGHT

A digest provides a compact fingerprint that can be compared against a trusted expected digest to detect a change.

Compact Operational Model

h = H(M)

M = message/data H = hash function h = digest

No SECRET KEY

Think Like an Engineer

A 5 GB container image changes by one bit. How can we efficiently determine whether it still matches the expected artifact?

A cryptographic hash is an unkeyed function that maps arbitrary-length data to a fixed-length digest.

Preimage Resistance

(emphasis: ANY input)



Classical Search Cost

$Pr[H(M)=h] \approx 1/2^n$

Classical brute-force scale: ≈ 2^n evaluations

Concrete Example — SHA-256

$n = 256$

Preimage search ≈ 2^{256} evaluations

Strong Hash ≠ Strong Password

$h = SHA256(P)$ vs. $SHA256(P_1), SHA256(P_2), SHA256(P_3), \ldots$

Human passwords come from a much smaller, more predictable candidate space.

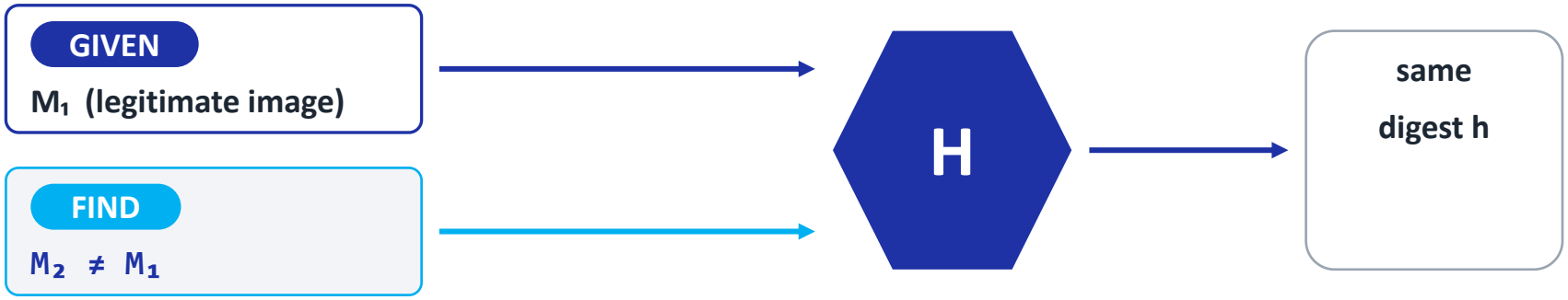
Think Like an Engineer

If SHA-256 has strong preimage resistance, why can SHA-256(password) still be vulnerable to guessing?

Preimage resistance makes finding an input for a specified digest computationally infeasible.

Second-Preimage Resistance

$H(M_2) = H(M_1) = h$



Engineering Example

Given: legitimate software image M_1

Attacker seeks a different image M_2 with $H(M_2) = H(M_1)$.

Classical Search Scale

Ideal n-bit hash: ≈ 2^n evaluations

SHA-256 example: ≈ 2^{256} evaluations

Think Like an Engineer

A trusted digest identifies a specific legitimate software image. Why should an attacker not be able to substitute a different image while preserving that digest?

Second-preimage resistance makes it infeasible to replace a specific message with a different message having the same digest.

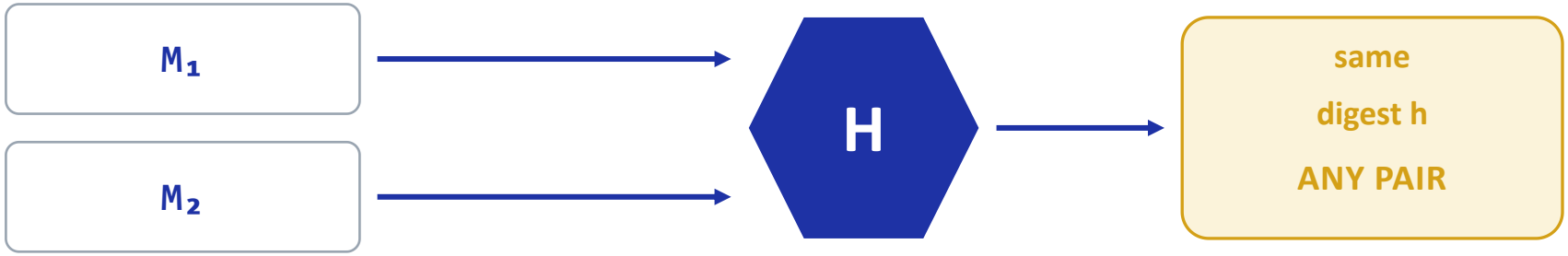
Collision Resistance

GIVEN

No specific message or digest

FIND

any $M_1 \neq M_2$ with $H(M_1)=H(M_2)$



Why Collisions Must Exist

Unbounded message space → H → only 2^n digest values

Different messages must eventually map to the same digest — collision-free hashing over arbitrary-length inputs is mathematically impossible.

Think Like an Engineer

If collisions must mathematically exist, how can a cryptographic hash function still be secure? Security requires collisions to be computationally infeasible to find — not nonexistent.

Collision resistance protects against attackers deliberately finding two different inputs with the same digest.

Birthday Attack: Why Collisions Are Easier

Birthdays (familiar analogy)

365 possible birthdays

Only 23 people needed

P(any two share a birthday) > 50% — looking for ANY TWO matching people, not one specific birthday.

≈

Cryptographic Hashes

$N = 2^n$ possible digests

Collision likely after √N samples

Looking for ANY matching pair $H(M_i)=H(M_j)$, $i \neq j$ — not a match to one predetermined digest.

$\sqrt{2^n} = 2^{n/2}$

Classical collision search: ≈ $2^{n/2}$ samples — not 2^n

SHA-256 (n = 256)

Preimage / Second-Preimage ≈ 2^{256}

Classical Ideal-Hash Security

Collision ≈ 2^{128}

Think Like an Engineer

Why does a 256-bit hash provide about 128 bits of classical collision security — not 256 bits? Collision search looks for any matching pair, and the birthday effect makes a pair likely after ≈ $2^{n/2}$ samples.

Collision security is fundamentally lower than preimage security for the same hash-output length.

Three Security Properties of a Cryptographic Hash

Preimage Resistance

GIVEN

h

↓

FIND

M such that $H(M) = h$

↓

SEARCH SCALE

$\approx 2^n$

Second-Preimage Resistance

GIVEN

M_1

↓

FIND

$M_2 \neq M_1$ with $H(M_2) = H(M_1)$

↓

SEARCH SCALE

$\approx 2^n$

Collision Resistance

GIVEN

Nothing specific

↓

FIND

$M_1 \neq M_2$ with $H(M_1) = H(M_2)$

↓

SEARCH SCALE

$\approx 2^{n/2}$ *Why?*

An n-bit hash has 2^n possible digest values — the possible input space is vastly larger.

Think Like an Engineer

An attacker is given one legitimate software image and tries to create a different image with the same digest. Which property is being challenged? (Expected answer: second-preimage resistance)

A secure cryptographic hash must make preimage, second-preimage, and collision attacks computationally infeasible.

HASH EVOLUTION

Evolution of Cryptographic Hash Functions

MD5

1992

128-bit digest

Collision resistance broken

SHA-1

1995

160-bit digest

Collision resistance broken

SHA-2

2001 onward

SHA-224 | SHA-256
SHA-384 | SHA-512

Modern — widely deployed

SHA-3

2015

SHA3-224 | SHA3-256
SHA3-384 | SHA3-512

Modern — different design

SHA-3 does NOT mean SHA-2 is broken.

SHA-2 remains secure and widely used. SHA-3 provides a different standardized design and additional design diversity.

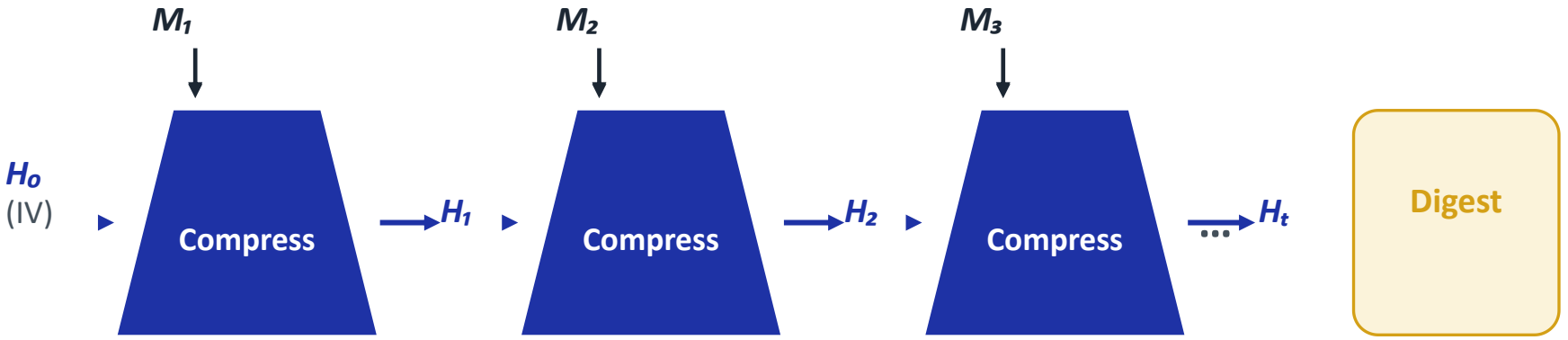
Think Like an Engineer

Does the existence of SHA-3 mean SHA-2 is broken? No — SHA-2 and SHA-3 use fundamentally different internal design approaches. We examine these next.

Hash standards evolve both in response to cryptanalysis and to provide strong, diverse design alternatives.

SHA-2: Iterative Hash Construction

$M \rightarrow$ Padding / Block Formation $\rightarrow (M_1, M_2, \ldots, M_t)$



$$H_i = \text{Compress}(H_{i-1}, M_i)$$

SHA-256
512-bit blocks → 256-bit digest

SHA-512
1024-bit blocks → 512-bit digest

ENGINEERING INSIGHT

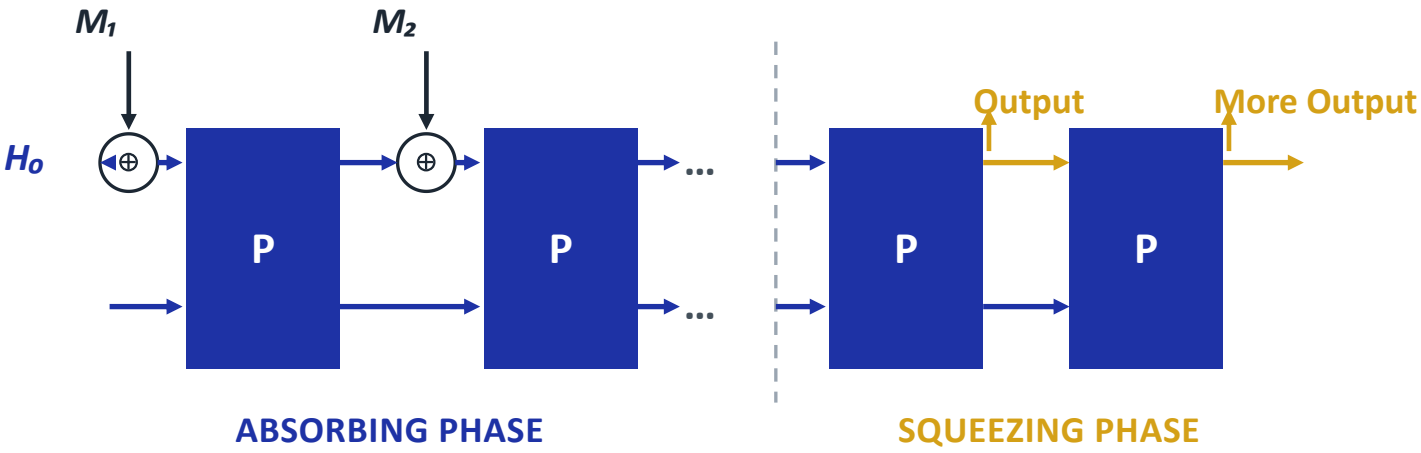
An arbitrary-length message is processed incrementally, block by block — not held in the hash state all at once.

Think Like an Engineer

How can a hash function process an arbitrarily long message while maintaining only a fixed-size chaining state?

SHA-2 converts arbitrary-length input into a fixed-length digest while processing only one fixed-size message block at a time.

SHA-3: Sponge Construction



Internal state b = r + c (r = rate, c = capacity) — input/output touches the rate; the permutation acts on the complete state.

SHA-2
Compression function + chaining state

SHA-3
Sponge construction + permutation

ENGINEERING INSIGHT

SHA-3 was not standardized because SHA-2 was broken. Both are modern standardized hash families — the point is design diversity.

Think Like an Engineer

Why standardize a fundamentally different hash design when SHA-2 remains secure? To provide a strong alternative and design diversity.

SHA-3 provides a modern hash family based on a fundamentally different internal architecture from SHA-2.

Using Hash Functions Correctly

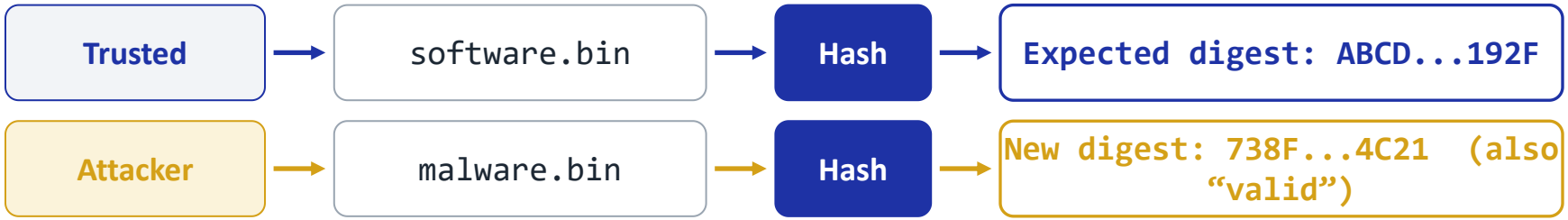
Appropriate Uses

- ✓ Data / content fingerprinting
- ✓ Integrity check vs. a trusted digest
- ✓ Digital-signature constructions
- ✓ Building block in larger protocols
- ✓ Hash-linked data structures

A Plain Hash Does NOT Provide

- ✗ Confidentiality (hashing ≠ encryption)
- ✗ Source authentication (no secret key)
- ✗ Secure password storage by itself
- ✗ Protection when data AND digest can both be replaced

Central Attack Example — Replacing Both File and Digest



SHA-256 has not failed — the attacker computed a valid digest for the replacement data. The failure: the expected digest was not authenticated.

Think Like an Engineer

If an attacker can replace both a downloaded file and its published hash, what security has the hash alone provided? — None: it never authenticated which file was intended.

A digest is meaningful for integrity checking only when the expected digest itself is trusted.

APPLIED CONNECTION

Hash Functions in Real Systems

Software & Container Integrity

Software / Image

↓

H( )

↓

Artifact Digest

AI-Model Integrity

Model / Checkpoint

↓

H( )

↓

Model Digest

Digital Signatures

Message M

↓

H( )

↓

Signature Mechanism

Blockchains

$Block_i$

↓

H( )

↓

⊃ $H(Block_{i-1})$

ENGINEERING INSIGHT — THE RECURRING PATTERN

Large object → compact fingerprint (hash) → trusted mechanism gives the fingerprint security meaning.

Think Like an Engineer

Across these applications, where does trust in the expected digest come from?

Possible answers: signed release metadata · authenticated repository/registry · digital signature + trusted public key · certificate/PKI · protocol validation/consensus

Hashes identify content; trusted protocols, signatures, or metadata determine what that identity means securely.

LECTURE SUMMARY

Cryptographic Hash Functions — Key Takeaways

1 — The Primitive

H maps {0,1}* → {0,1}^{n}

Variable → fixed digest

No secret key

2 — Three Properties

Preimage

2nd Preimage

Collision

Differ: GIVEN vs. FIND

3 — Security Scale

Preimage: ≈2^n

2nd-Preimage: ≈2^n

Collision: ≈2$^{n/2}$

(birthday effect)

4 — Modern Families

SHA-2

compression + chaining

SHA-3

sponge + permutation

5 — Secure Use

Not encryption

Not source auth

Not for passwords

Needs trusted digest

If a cryptographic hash has no secret key, how can hash-function machinery be used for authentication?

Next → MAC and HMAC

From fingerprints to keyed authentication